Can AI crack the code of a system nobody understands

MortisAI-Lugano26
Lifeware hackathon report

Abstract

We recover the calculation rules of a legacy insurance system and write executable formulas from its input-output examples. Visual analysis exposed historical changes in rounding, age conventions and timing, which led us to describe each calculation as a known business formula with open decisions. We use program synthesis to combine reusable building blocks into candidate formulas and verify their outputs through exact execution. When search stalls, the user can optionally invoke a self-hosted, open-weight LLM to propose missing building blocks. We also develop an alternative in which a pretrained MLP guides formula generation without any LLM calls. Both approaches produced accurate executable formulas in their respective evaluations: insurance calculations for the vocabulary-based solver and held-out synthetic tasks for the MLP-guided engine.

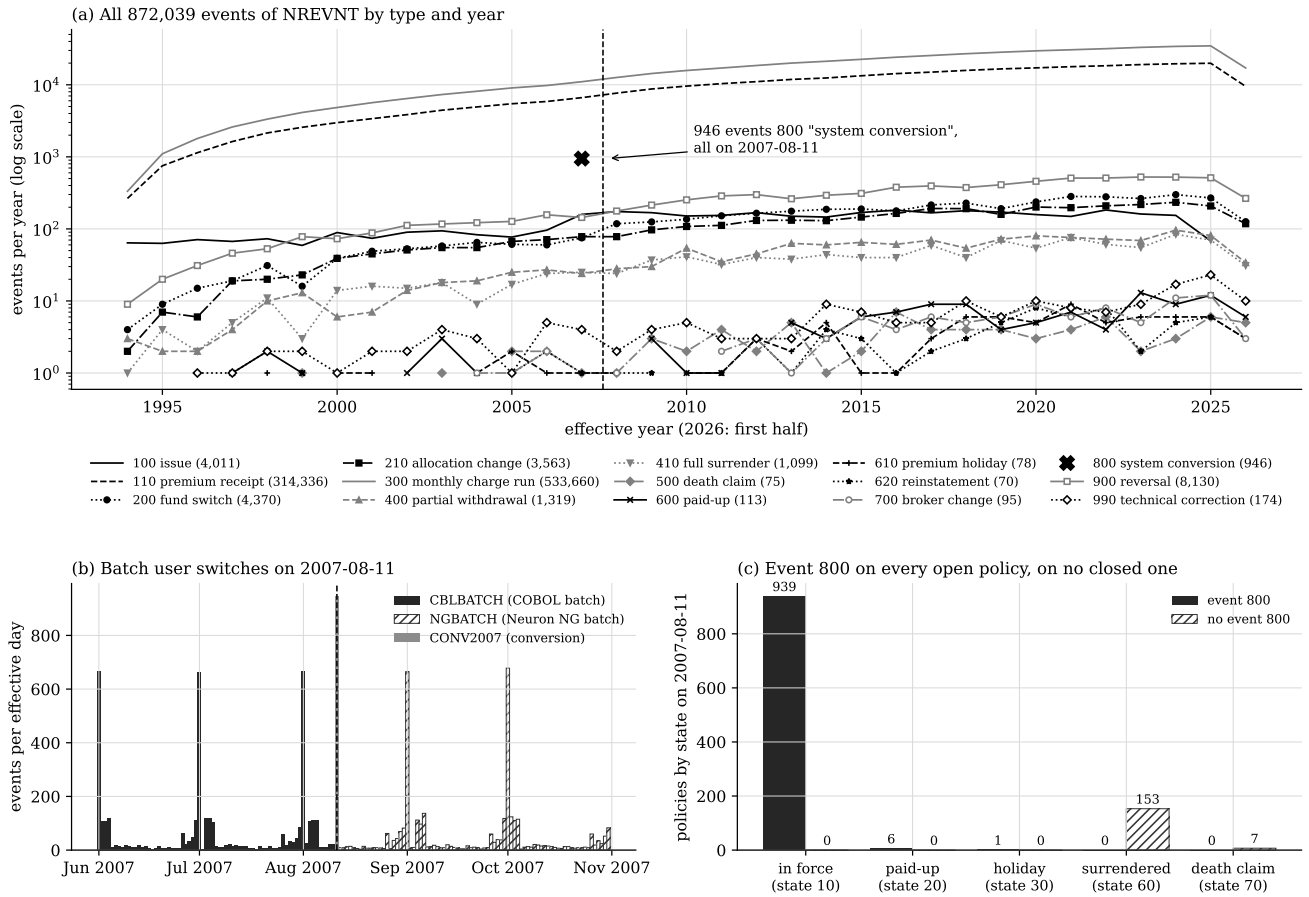


Figure 1: Event history of the Neuron extract (`NREVNT`). (a) All 872,039 events by type and year, log scale. The portfolio grows smoothly through the 2007 system change (dashed line); the only discontinuity is a one-day event: 946 events of type 800 ("system conversion"), all dated 11 August 2007. (b) Events per effective day around that day: the monthly batch user switches from `CBLBATCH` (COBOL) to `NGBATCH` (Neuron NG) on 11 August 2007. (c) Policies by state on 11 August 2007: every open policy (in force, paid-up, premium holiday; 946) received an event 800, and none of the 160 closed ones (surrendered, death claim) did. Visual exploration of all events thus located the system change that later became the era cut-off of every rule.

1 From visual exploration to a search problem

The challenge provides data from an old insurance system and examples of its calculations. These include commissions, payment dates, maturity values, surrender values and annual statements. The data contains 27 input tables, 4,011 policies and about 6.8 million bookings. Accurate calculations are important for this domain. Even a one-cent error makes the whole output row incorrect. We built a process that repeats several steps. First, we read and connect the relevant tables. Then we generate possible formulas and compare their results with the known answers. We study the errors and use them to improve the search. The goal is to find a formula that can run as code and whose steps are easy to check.

2 Inductive program synthesis

We reviewed numerical and symbolic regression, function approximation, and program synthesis. Symbolic regression searches for readable expressions; divide-and-conquer synthesis combines expressions that solve different subsets of examples. Our early solutions struggled with the stateful accounting required by some tasks. This motivated explicit building blocks for dates, historical lookups, arithmetic and rounding.

2.1 Formulas with holes

Most Neuron rules turned out to be a known business formula with a few small open decisions: how to round, which calendar, the price or exchange rate of which day, which system did the calculation. We call these decisions *holes* and their candidate answers *bricks*. The *manually supplied structure* is the skeleton of each calculation, written once per system type: the commission formula (t0), the payment schedule (t1), the monthly offer projection (t2) and the unit ledger replay (t3–t4). Everything inside the skeletons is a hole. We listed 72 conventions this way, each with alternatives.

The vocabulary contains generic families rather than single answers: all roundings to 0–6 digits (half-up, truncate, half-even, up, 5 Rappen), six age rules, Swiss and Ticino calendars with Easter, business-day shifts of ±5 days, day counts and growth rules. It now spans 39 convention modules. The vocabulary was extended while the hand-written solution was known; we counter this with generic families and with ablations (Sec. 2.3).

2.2 Search and validation

Exact match. A candidate is scored by the share of rows that match Neuron to the cent and the day, not by mean error. **Policy-level split.** 20 % of the policies take no part in the search; the winner must also hold on them. **Large spaces.** For t2 (14 holes, 48 orderings of a month) and t3 the engine varies one hole at a time and two when it gets stuck; one t2 sweep is 125 probes on 10 % of the policies (≈70 s). **Ties.** Equal options are resolved by consistency with the other rules of the same era, then by business meaning, then by simplicity. Ties decided by simplicity are re-checked on all training policies: on a 3 % sample two deduction rules looked tied, while the full recomputation separated them (1,077,101 of 1,077,102 deductions). **Tree on misses.** If one rule leaves misses, a one-level tree searches for a split. It accepts a split directly only at a *business* boundary (a one-day event shared by many policies, the start of a parameter row, 1 January); otherwise the LLM is asked first. **Output.** The choice and its evidence (rows explained, held-out result, losing options, era) are written to `rules.py`. Two runs give a byte-identical file, and a quality gate fails if `rules.py` is edited by hand.

The system conversion is found by this loop. One rule explains 73 % of the offers; the tree cuts at 2007-08-11, the day of all 946 conversion events (type 800, batch user `CBLBATCH`→`NGBATCH`). Before it: truncation and age = ⌊days/365⌋ (Fig. 1); after it: half-up and age at nearest birthday, 100 % on both sides. At run time the solution reads the date from the type-800 events of each extract.

2.3 The LLM in the loop

We run Qwen3.8-27B (BF16) on one rented RTX PRO 6000 (96 GB, vLLM, MTP speculative decoding, ≈65 tokens/s, about $1.4/h). The model is called only when the search is stuck. Its prompt holds the best rule so far, the missed rows *next to correctly predicted rows*, and the bricks already tried; it must return one Python function. The function passes an AST sandbox (no imports) and is accepted only if held-out accuracy rises. Accepted bricks are stored with provenance, so later runs reproduce the result without the LLM. The submitted code never calls an LLM, and the layer is off by default (flag `-llm`).

Ablations. With Easter removed from the vocabulary, t1 falls to 94 %. Qwen's first brick also added Good Friday, a holiday in most cantons but not in Ticino, and was rejected; once correct rows were shown beside the misses, its brick restored 100 % on search and held-out policies. With the days/365 age rule removed, Qwen rediscovered it (100 %). One ablation exposed a weakness: without the anchor-day rule, 99.95 % of t1 still matched, above our 99.9 % threshold, and the engine labelled four policies as manual corrections without asking the LLM. It now asks whenever misses concentrate on one feature.

2.4 Evaluation

All accuracies come from the official `check.py` on outputs of a *complete history replay*: t3 rebuilds every unit purchase and monthly deduction from the first booking. Intermediate-state diagnostics (each of 407,636 purchases and 1,077,102 deductions checked separately) were used only to localise errors during search.

Task	Exact rows (train)	Accuracy
t0 commission	225,848 / 225,858	99.996 %
t1 payment dates	15,028 / 15,028	100 %
t2 projection	3,011 / 3,019	99.735 %
t3 surrender value	6,137 / 6,153	99.740 %
t4 statements	34,124 / 34,145	99.939 %

Table 1: End-to-end accuracy of the generated rules. All 55 remaining misses are manual corrections in Neuron, flagged by t5. The public leaderboard policies score 100 % on t0–t4.

The engine finds 65 of the 72 conventions without a human in the loop; the rest are method parameters or have no data to test them. A full `autosolve` run takes about 13 minutes. The work was planned at 30–50 hours; it took about 6.5.

New clients. On unseen synthetic clients (Sec. 4) the fixed solution dropped sharply, and so did `autosolve` at first, because it silently assumed our client in two places. After the fix, one run recovered the hidden rules, including a conversion date of 2010-07-03 (Table 2).

Client	Version	t0	t1	t2	t3	t4
L2	fixed rules	96.3	100	90.8	65.7	58.7
L2	autosolve	100	100	100	99.9	100
L3	fixed rules	97.2	92.3	0.0	23.0	0.8
L3	autosolve	100	99.8	100	90.4	99.5

Table 2: Accuracy (%) on test policies of unseen synthetic clients.

2.5 Surprise task t7: rules without answers

The tax audit came without local answers; the leaderboard scored a few rows every two hours, and the privacy rule allowed only our own open-weight model on the tax data. We replaced targets with checkable evidence: the `NRTAXB` bookings (no amounts, but who is taxed, when, by which tax, in which country), both worked examples of the statement (reproduced to the cent), consistency between payout and annual tables, and *bitemporal* versions, since Neuron only saw addresses and rates entered before the calculation. Each submission tested one hypothesis with its outcomes written down beforehand. A first score of 1/20 did not move when we changed the base of untaxed rows, which pointed to something shared by all untaxed rows: a missing remittance date was emitted as `None` and dropped. An empty string gave 11/20.

When two samples (110 rows) were released, they became

targets. Mismatches were reduced to *fingerprints*: wrong column, ratio answer/ours, difference in cents or days. An SOS ratio of 0.625 = 12.5 %/20 %, for example, revealed that Italian gains are split by rate period. Eleven conventions followed, among them national tax-office holidays, the German 15 % fund exemption *not* applied, truncated SolZ and KiSt, and premiums rounded with the rules of their own day. They enter `autosolve` as bricks of the t7 case, a variant is kept only if it is no worse on either sample and better overall (six rules are supported by one sample only). Result: 110/110 on the released samples (from 64/110) and X/Y on the unseen third sample.

3 MLP-guided engine

We developed a self-contained formula generator guided by an MLP, inspired by . Typed arithmetic, calendar and conditional operations formed candidate expressions; the MLP ranked partial formulas to decide which fragments to extend. Exact Decimal execution checked candidates, and accepted expressions were exported as readable Python functions.

The design offered three technical advantages. **Autonomy:** trained weights and search ran locally, without runtime LLM calls or an external inference service. **Transferability:** the ranker used relationships between inputs, intermediate values and outputs rather than policy identifiers, allowing reuse on new compositions within the supported vocabulary. **Trainability:** synthetic programs and difficult candidates collected during search provided further supervision. Learned guidance and deterministic execution remained separate, keeping the resulting calculations inspectable.

The scaled 10M-parameter prototype (trained for 26 epochs) was benchmarked against baseline heuristics across 120 held-out tasks under a strict 1.0-second time budget per attempt: the heuristic baseline, 10M MLP, and hybrid setup verified solutions on 7/120 tasks, compared to 8/120 for the compact checkpoint. While initial gains remain marginal under aggressive latency constraints, extended training exhibits strong latent potential once representations converge. Consistent with property-signature guided synthesis in BUSTLE, neural heuristics require sufficient training exposure and tighter integration with enumerative search to overcome inference overhead. Rather than discarding capacity scaling, realizing its benefit hinges on longer optimization schedules, richer feature encoding, and domain-specific search pruning.

4 Testing the method and the evidence

A bench of unseen clients. We built a generator of synthetic clients in levels: L1 replays our client, L2–L3 move rates, dates and calendars, L6 adds conventions absent from the original data (5-Rappen rounding, 30E/360, age by 365.25 days, half-down, the Vaud calendar, modified following), and L7 runs L6 on edge cases (month-end and 29 February starts, reversals, a conversion on the busiest day). Every client passes 35 automated checks, e.g. no original policy number and no trace of the original conversion date. The bench reproduces `check.py` on the original data, and a second reference written from the task text alone matched t0 and the t4 premiums row by row on five clients (1,131,909 commissions, 165,809 statement rows). Mutation testing caught 23 of 27 deliberately broken references; the four survivors are one-day boundary errors that no client exercises.

Harder clients. On L6 `autosolve` reached 100, 99.9, 100, 94.5 and 93.4 % (t0–t4), on L7 100, 99.6, 93.0, 92.8 and 94.0 %, where the fixed rules score 0–1 % on t2 and t4. Across L2–L7 it never chose a wrong rule: every miss was a convention missing from the vocabulary. Without any answers, metamorphic tests (renamed policies, 30 % of policies removed, shuffled rows, padded fields, an extra column) left every output unchanged.

What the t7 leaderboard could tell. The 21 open t7 decisions span $4 \cdot 3^5 \cdot 2^{16} \approx 6.4 \cdot 10^7$ combinations (25.9 bits); one upload of n rows reveals at most $\log_2(n+1)$ bits, 4.4 for 20 rows. A binomial test flagged the change of the scored sample ($p \approx 0.001$) before it was announced. An influence map counted the rows each alternative changes (up to 1,813 payout and 1,517 annual rows) and set the order of uploads. A law-only model already reproduced 87 % of our payout rows, and the law agreed with `NRTAXB` on 7,476 of 7,479 tax decisions: the open part was conventions, not law.

Every t7 rule is needed. The eleven rules are switches in `rules.py`; with the defaults the output is byte-identical to the submitted build. Switching off any single rule lowers the sample score (Table 3), e.g. rate periods: sample 2 annual 30/30→19/30; loss base 0: 395 payout rows of the extract change.

t7 build	Sample 1	Sample 2	Rows
before the samples (#68)	26/40	33/70	59/110
+ five earlier choices	32/40	45/70	77/110
+ SOS rate periods, tax year	35/40	55/70	90/110
+ calendars, stamp base, Germany	35/40	61/70	96/110
+ earlier withdrawals, TAXFIX19	39/40	65/70	104/110
+ premium rounding, death benefit	**40/40**	**70/70**	**110/110**
final build without one rule			
SOS rate periods	36/40	56/70	92/110
tax year = remittance year	39/40	65/70	104/110
premium rounding by its day	40/40	65/70	105/110

Table 3: t7 on the two released samples (payout + annual rows). Each step adds rules found from error fingerprints; removing any single rule lowers the score.

A private open-weight LLM. We deployed our own open-weight model, Qwen3.8-27B, on a dedicated GPU server under our control: prompts and data never reach a third-party service, which also meets the privacy requirement of the tax task. The solution works without it. The model is an optional layer of `autosolve`: with `-llm`, whenever the vocabulary cannot explain part of the data, for example on an unseen client or a new quantity, Qwen proposes the missing brick, and the brick is kept only if held-out accuracy rises. In ablation tests we removed rules from the vocabulary (Easter holidays, the COBOL age rule) and Qwen rebuilt them, bringing t1 back from 94 % to 100 % (Sec. 2.3). On our client the vocabulary already covers every convention, so the final run is made without the flag, and the submitted code never calls an LLM.